

1.1操作系统的基本概念



1.2操作系统的发展与分类



1.4大内核与微内核

大内核

将操作系统的主要功能模块进行集中，从而用以提供高性能的系统服务

优点：各个管理模块之间共享信息，能够有效利用相互之间的有效特性，所有有着巨大的性能优势

缺点：层次交互关系复杂，层次接口难以定义，层次之间界限模糊

背景：随着计算机体系结构的不断发展，操作系统提供的服务越来越多,接口形式越来越复杂

将内核中最基本的功能（如：进程管理）保留在内核，将不需要在核心态执行的功能转移到用户态执行，降低内核设计的复杂性

微内核

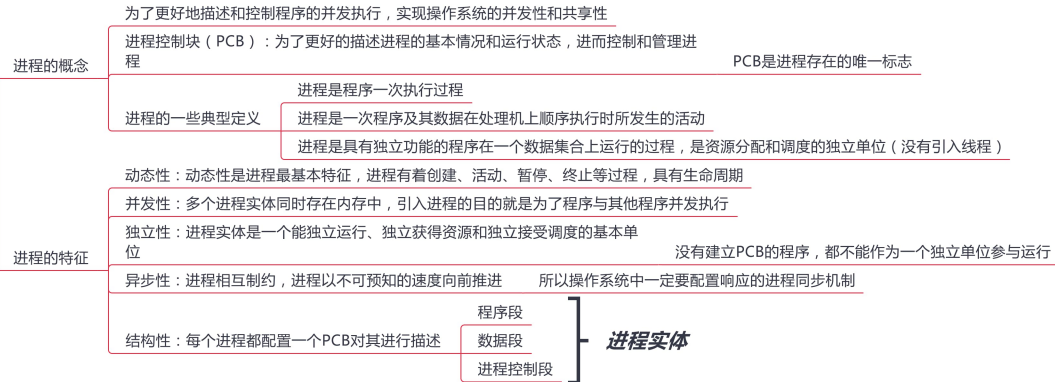
优点 有效的分离内核与服务、服务与服务、使得他们之间的接口更加的清晰，维护的代价大大降低

各部分可以独立的优化和演进

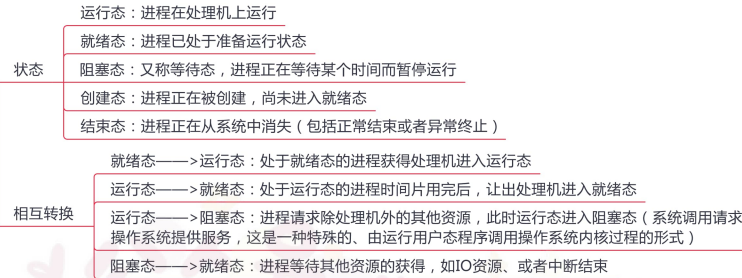
缺点 性能问题，需要频繁的在核心态和用户态之间进行切换

2.1进程与线程（上）

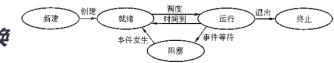
进程的概念和特征



进程的状态与转换



相互转换



进程控制



阻塞是一种自主行为，自我阻塞

唤醒是被相互有联系的其他进程进行唤醒

2.1进程与线程（中）

进程的组织

进程是一个独立的运行单位，也是操作系统进行资源分配和调度基本单位，由以下三部分构成

进程描述信息

进程标识符：标志进程

用户标识符：进程归属的用户，主要为共享和保护服务

进程当前状态：描述进程状态信息

进程优先级：描述进程抢战处理机优先级

代码运行入口地址

程序的外存地址

进入内存时间

处理机占用时间

信号量使用

进程控制和管理信息

资源分配清单

用以说明有关内存地址空间或者虚拟地址空间状况，所打开的文件的列表和所使用的的输入/输出设备信息

代码段指针、数据段指针、堆栈段指针、文件描述符、键盘、鼠标

处理机相关信息

处理机中各寄存器的值

通用寄存器值、地址寄存器值、控制寄存器值，标志寄存器值，状态字

程序段：能被进程调度程序调度到CPU执行的程序代码段

数据段：进程对应的程序加工处理的原始数据或者程序执行时产生的中间或者最终结果

进程的组织方式

链接方式

按照进程状态将PCB分为多个队列

操作系统持有指向各个队列的指针

索引方式

根据进程状态的不同,建立几张索引表

操作系统持有指向各个索引表的指针

进程的通信

共享存储

通信进程之间存在一块可以被直接访问的共享空间

低级方式：基于数据结构共享

高级方式：基于存储区共享

操作系统只负责为通信进程提供可共享使用的存储空间和同步互斥工具，数据交换则由用户自己安排读/写指令完成

消息传递

进程间的数据交换是以格式化的消息为单位的，进程通过系统提供的发送消息和接收消息的两个原语进行数据交换

直接通信方式：发送进程直接发送消息给接收进程，并将它挂在接收进程的消息缓冲队列上，接收进程从消息缓冲队列中取得消息

间接通信方式：发送进程把消息发送给某个中间实体，接收进程从中间实体中获得消息 例如电子邮件系统

管道通信

发送进程以字符流形式将大量数据送入写管道，接收进程从管道中接收数据

当管道写满时，写进程的write()系统调用将被阻塞，等待读进程将数据取走

当读进程将数据全部取走后，管道变空，此时读进程的read()系统调用将被阻塞

功能：互斥、同步、确定对方存在

半双工通信，不可以同时读和写

限制管道的大小

管道变空的时候阻塞读进程

管道中的数据被读取后会马上被丢弃

线程概念和多线程模型

线程基本概念

减小程序在并发执行时所付出的时空开销，提高操作系统的并发性能

引入线程后，进程只作为系统资源的分配单元，线程作为处理机的分配单元

线程与进程的比较

调度 传统中进程是资源和独立调度的基本单位

引入线程后，进程是独立调度的基本单位，线程是资源的基本单位

不同进程的线程切换会引起进程切换

拥有资源 进程是资源分配的基本单位

并发性 引入线程后，进程可以并发执行，多个线程之间也可以并发执行，提高了系统的吞吐量

系统开销 同一进程的线程切换要比进程切换开销小的多

地址空间和其他资源 进程的地址空间之间相互独立，统一进程的各线程之间共享进程的资源，某进程的线程对其他进程不可见

通信方面 进程间通信需要进程同步和互斥手段的辅助，保证数据的一致性

线程间可以直接读/写进程程序段来进行通信

线程属性

不拥有系统资源，拥有唯一标识符和线程控制块

不同的线程可以执行相同的程序，同一个服务程序被不同用户调用时，操作系统将其创建为不同线程

统一进程的线程共享该进程拥有的全部资源

线程是处理机的独立调度单位

线程也有生命周期，阻塞，就绪，运行等状态

多CPU计算机中,各个线程可占用不同的CPU

每个线程都有一个线程ID、线程控制块（TCB）

切换同进程内的线程,系统开销很小

切换进程,系统开销较大

由于共享内存地址空间,同一进程中的线程间通信甚至无需系统干预

2.1进程与线程（下）

线程的实现方式

- 用户级线程 有关线程管理的所有工作都由应用程序完成，内核意识不到线程的存在
- 内核级线程 线程的管理工作全部由内核完成

多线程模型

- 多对一
 - 经多个用户级线程映射到一个内核级线程，线程管理在用户空间完成，用户级线程对操作系统不可见
 - 优点：线程管理是在用户控件进行的，效率比较高
 - 缺点：一个线程阻塞全部线程都会阻塞，多个线程不能并行运行在多处理机上
- 一对一
 - 每个用户级线程映射到一个内核级线程上
 - 优点：并发能力强
 - 缺点：创建线程开销大，影响应用程序的性能
- 多对多
 - 多个线程映射到多个内核线程上
 - 结合上述两种，既可以提高并发性，又适当的降低了开销

2.2处理机调度（上）



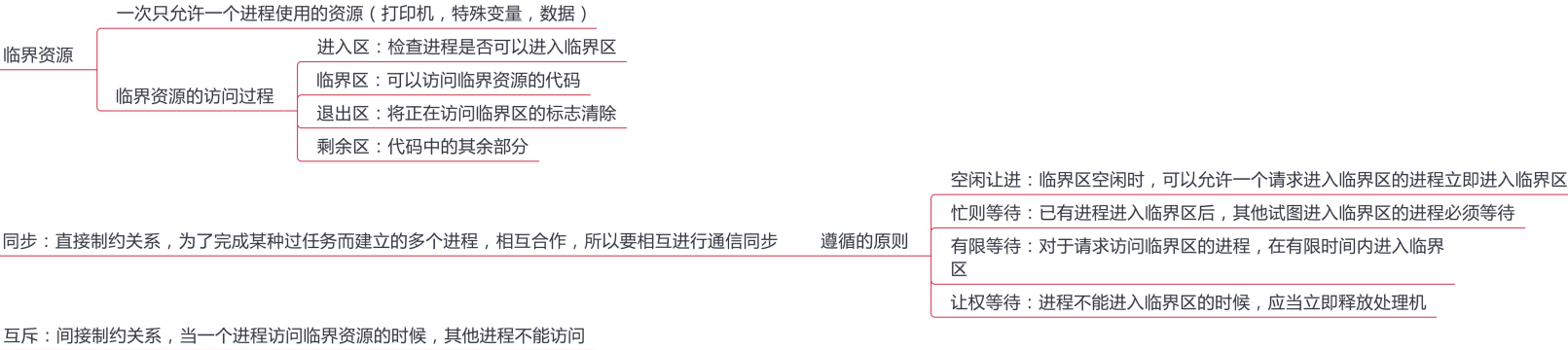
2.2处理机调度（下）

典型调度算法

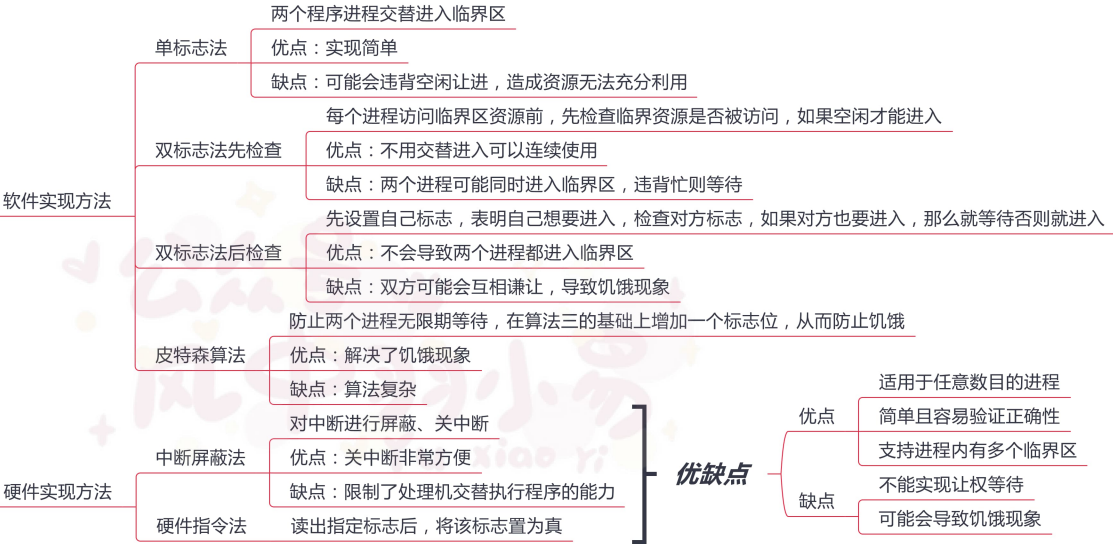


2.3进程同步

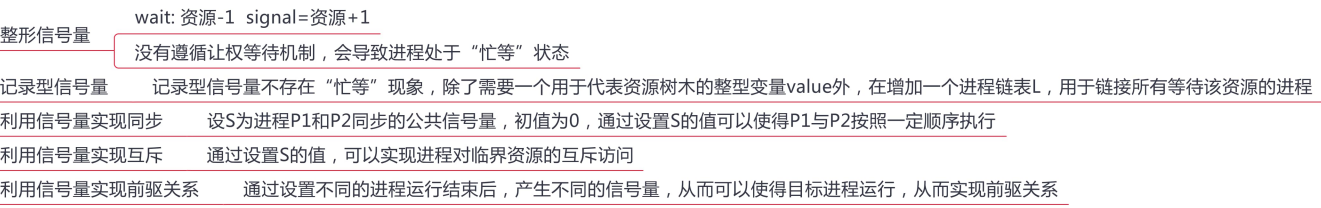
基本概念



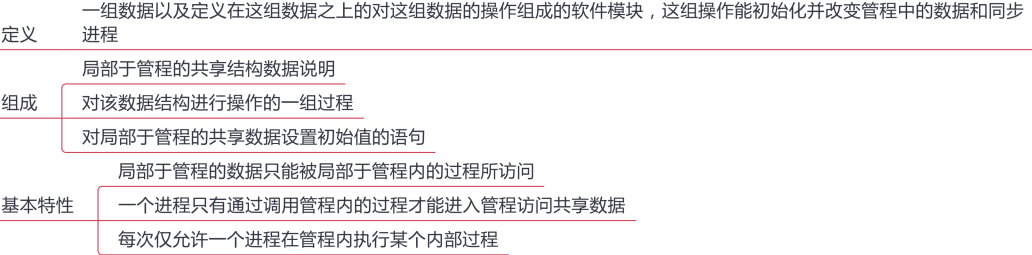
实现临界区互斥的基本方法



信号量



管程



2.4 死锁

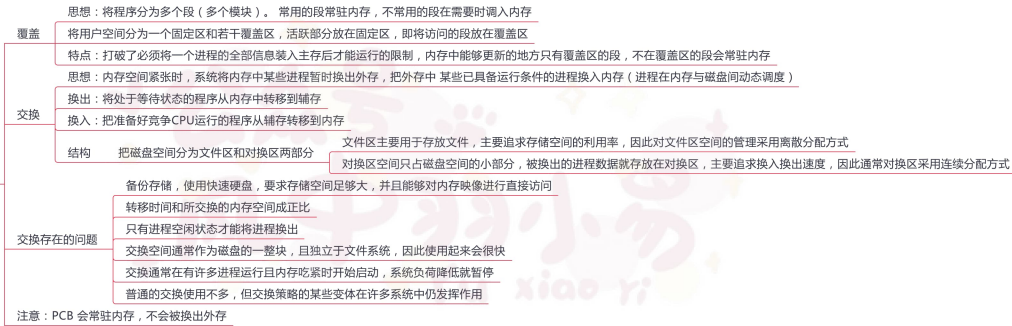


3.1 内存管理概念（上）

内存管理的基本原理和要求



覆盖与交换



连续分配管理方式



3.1 内存管理概念（中）

非连续分配管理方式

允许一个程序分散的装入不相邻的内存分区

设计思想

将主存空间划分为大小相等且固定的块，块相对较小，作为主存的基本单位，进程以块为单位尽心空间申请
分页存储与固定分区技术很像，但是其分页相对于分区又很小，分页管理不会产生外部碎片，产生的内部碎片也非常的小

分页存储的基本概念

进程中的块 = 页
页面和页面大小
内存中的块 = 页框（页帧）
进程申请主存空间，为每个页面分配主存中可用页框，即页与页框一一对应

页面大小要适中

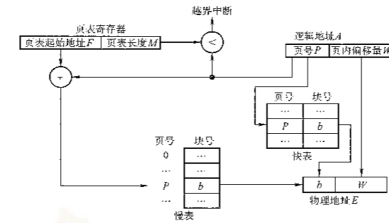
页面太小：进程页面数过多，页表过程，增加内存占用，降低硬件地址转换效率

页面太大：页内碎片过多，降低内存利用率

地址结构 页号（有多少页的编号）+ 页内偏移（页内存了多少东西）

页表 为了便于在内存中找到进程的每个页面对应的物理块，系统为每个进程建立一张页表，记录页面在内存中对应的物理块号，页表一般放在内存中

页表项：页号 + 物理内存中的块号（不要与地址结构搞混） 页表项的物理内存块号 + 地址结构中的页内偏移 = 物理地址



基本分页存储管理方式

基本地址变换机构

计算方式
页号 $P = A/L$ ，页内偏移量 $W = A \% L$
比较页号P和页表长度M，若 $P > M$ 产生越界中断
页表中页号P对应的页表项地址 = 页表起始地址F + 页号P * 页表项长度 取出该页表项内容b
计算 $E = b * L + W$ 使用E去访问内存

页表项大小的设计应当尽量一页正好能装下所有的页表项

地址变换过程必须足够快，否则访存速率会降低

分页管理存在的问题 页表不能太大，否则会降低内存利用率

组成
设置一个页表寄存器（PTR），存放页表在内存中的起始地址F 和页表长度M
页表的起始地址和页表长度放在进程控制块（PCB）中

可优化方向：如果页表放在内存中，取地址访问一次内存，按照地址取出数据访问一次内存，共需要两次访问内存

优化：地址变换机构中增加一个具有并行查找能力的高速缓冲寄存器（快表），又称为相联存储器（TLB） 相联存储器可以按照地址查找也可以按照内容查找

具有快表的地址变换机构

访问一个逻辑地址的访存次数 基本地址变换机构 两次访存

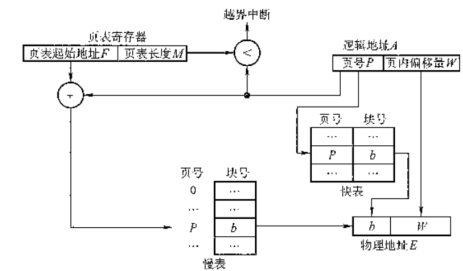
具有快表的地址变换机构
快表命中，只需一次访存
快表未命中，需要两次访存

变换过程 CPU给出逻辑地址后，先查询快表中是否命中

若快表命中，直接从快表中该页对应的页框好，与页内偏移量拼接成物理地址

若快表不命中，再按照正常方式从页表中查询相应页表项，并将该页表项存入快表中（按照一定策略）

查找图示



两级页表

如果页数过多，就会导致页表也过多，那么我们可以考虑设置一个用来储存页表的页表(套娃)

逻辑地址空间格式 = 一级页号 + 二级页号 + 页内偏移

设计多级页表的时候，最后一定要保证顶级页表一定只有一个

建立多级页表的目的在于建立索引，不必浪费主存空间去储存无用的页表项，也不用盲目式的查询页表项

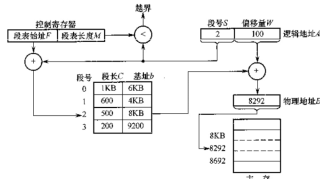
3.1 内存管理概念（下）

基本分段存储管理方式

- 出发点
 - 分页是从计算机角度考虑设计的，目的是为了内存的利用率，提高计算机性能，分页通过硬件机制实现，对用户完全透明
 - 分段是从用户和程序员的角度提出，满足方便编程，信息保护和共享，动态增长及动态链接等多方面的需要
- 分段
 - 按照用户进程中的自然段划分逻辑空间
 - 地址结构 = 段号S + 段内偏移量W
- 段表
 - 页式系统中，页号和页内偏移对用户透明 段式系统中 段号和段内偏移量必须由用户显示的提供
 - 每个进程都有一张逻辑空间与内存空间映射的段表，这个段表项对应进程的一段，段表项记录该段在内存中的起始地址和长度
 - 段表内容 = 段号 + 段长 + 本段在内存中的地址

- 地址变换机构
 - 逻辑地址A中取出段号S和段内偏移量W
 - 比较段号S和段表长度M，若S>=M,则产生越界中断，否则继续执行
 - 段号S对应的段表项地址 = 段表起始地址F + 段号S * 段表项长度，从该段表项中取出段长C，比较段内偏移量与C的大小判断是否出现越界
 - 取出段表项中该段的起始地址b,计算E = b + W，用得到的物理地址E去访问内存
- 段的共享与保护
 - 共享：两个作业的段表中响应表项指向被共享段的同一个物理副本来实现的 纯代码或者可重入代码以及不可修改的数据可以被共享
 - 保护机制
 - 存取控制保护
 - 地址越界保护

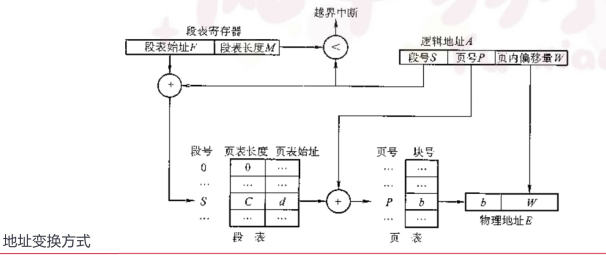
变换过程



页式存储有效的提高内存利用率，分段存储能反映程序的逻辑结构并有利于段的分享，将这两种方式结合一下 这种二者结合的方法经常在计算机理论中遇到

段页式管理方式

- 思想
 - 作业的地址空间首先被分成若干逻辑段，每段有自己的段号
 - 每个段分成若干大小固定的页
 - 对内存空间的管理仍然和分页存储管理一样
- 地址结构 段号S + 页号P + 页内偏移量W
- 为了实现地址变换，系统为每个进程建立了一张段表，每个分段有一个页表 一个进程中，段表只能有一个，页表可以有多个

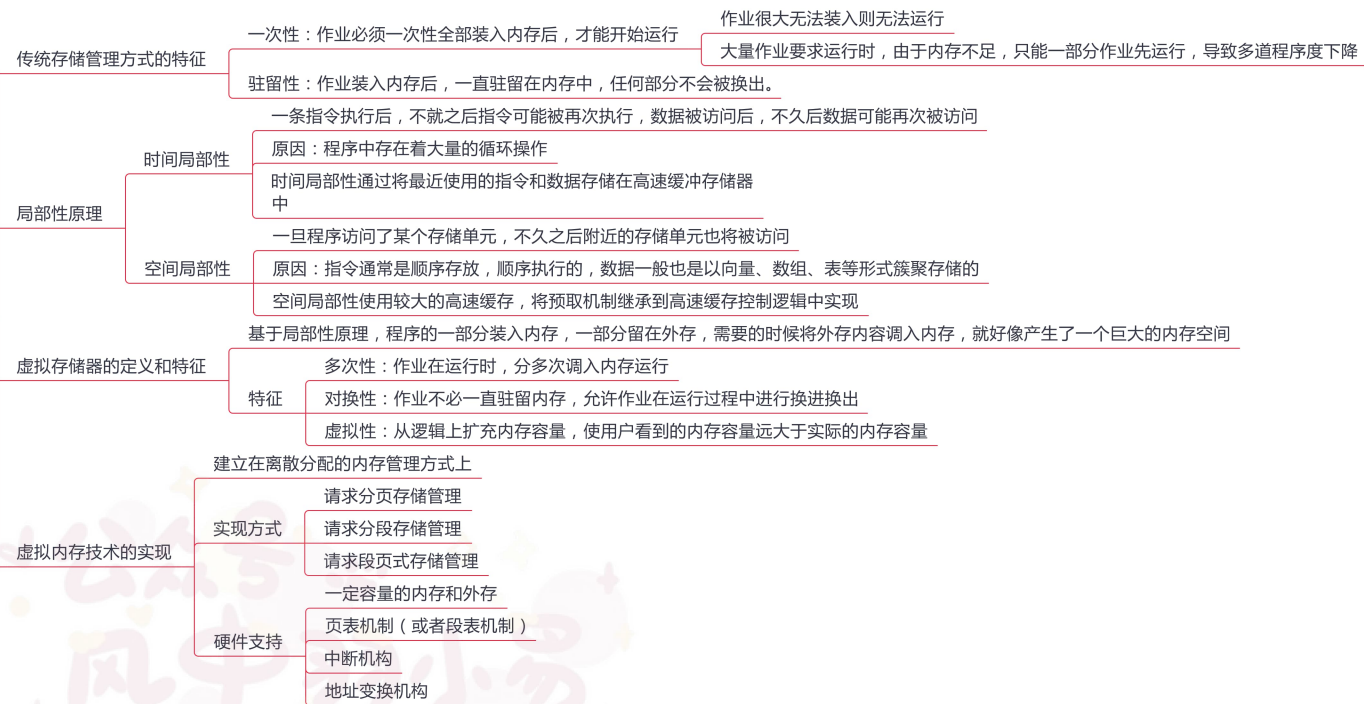


补充

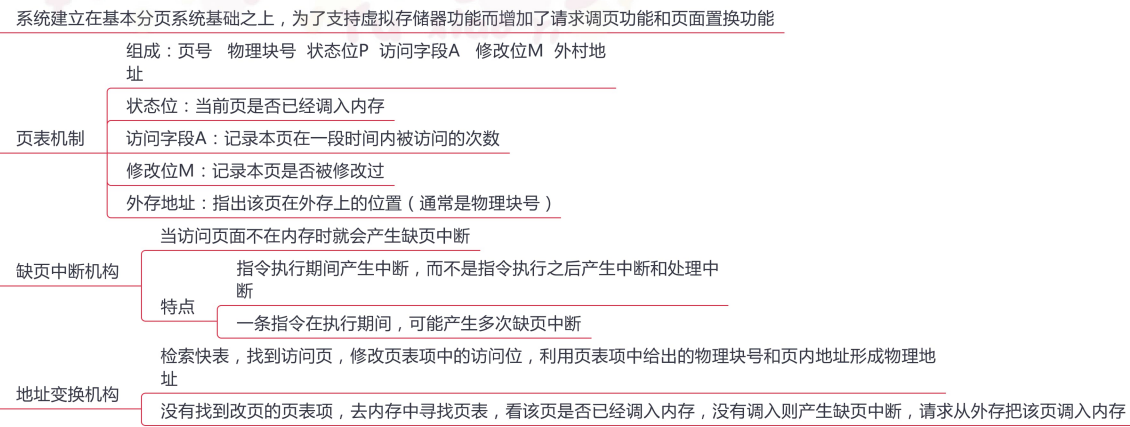
- 不能被修改的代码称为纯代码或可重入代码（不属于临界资源）
- 分段与分页的区别
 - 分页对用户不可见，分段对用户可见
 - 分页的地址空间是一维的，分段的地址空间是二维的
 - 分页（单级页表）、分段访问一个逻辑地址都需要两次访存，分段存储中也可以引入快表机构
 - 分段更容易实现信息的共享和保护（纯代码可重入代码可以共享）
- 分段与分页优缺点
 - 分页管理
 - 优点：内存空间利用率高，不会产生外部碎片，只有少量的页内碎片
 - 缺点：不方便按照逻辑模块实现信息的共享和保护
 - 分段管理
 - 优点 很方便按照逻辑模块实现信息的共享和保护
 - 缺点 如果段长过大，为其分配很大的连续空间会很不方便
 - 段式管理会产生外部碎片

3.2虚拟内存管理（上）

虚拟内存的基本概念



请求分页管理方式

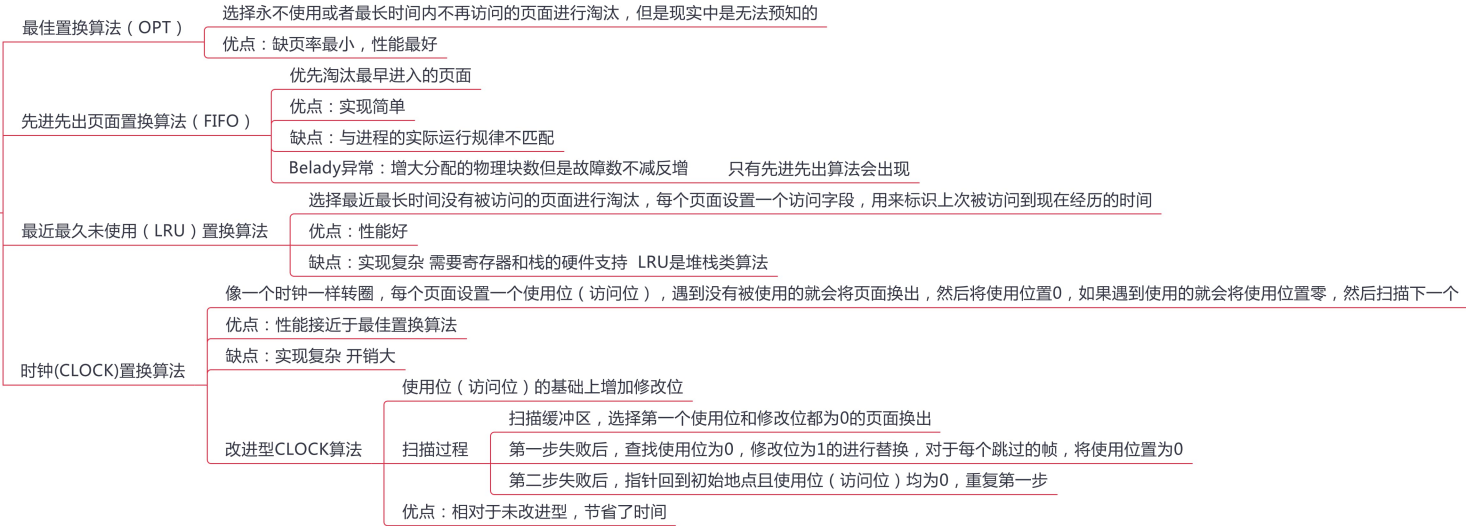


易混知识点

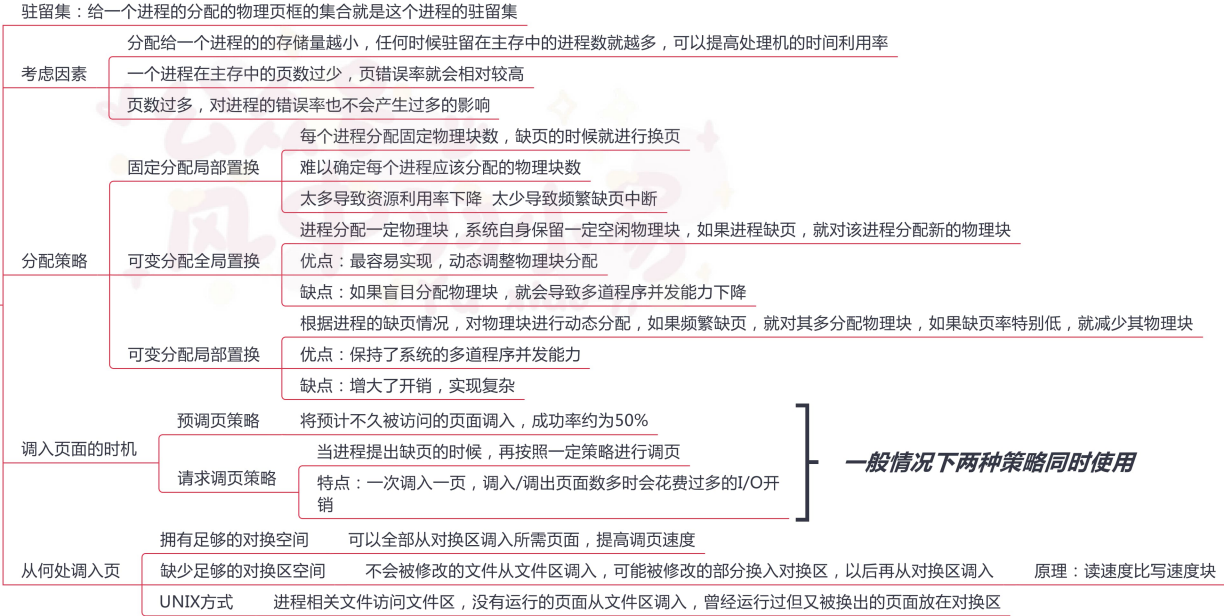
- 虚拟内存的最大容量是由计算机的地址结构（CPU寻址范围）确定的
- 虚拟内存的实际容量 = min（内存和外存容量之和，CPU寻址范围）

3.2虚拟内存管理（下）

页面置换算法

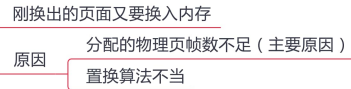


页面分配策略

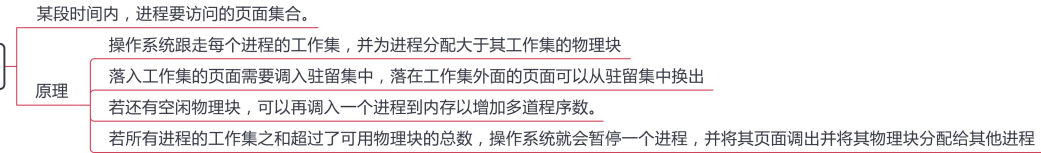


一般情况下两种策略同时使用

抖动

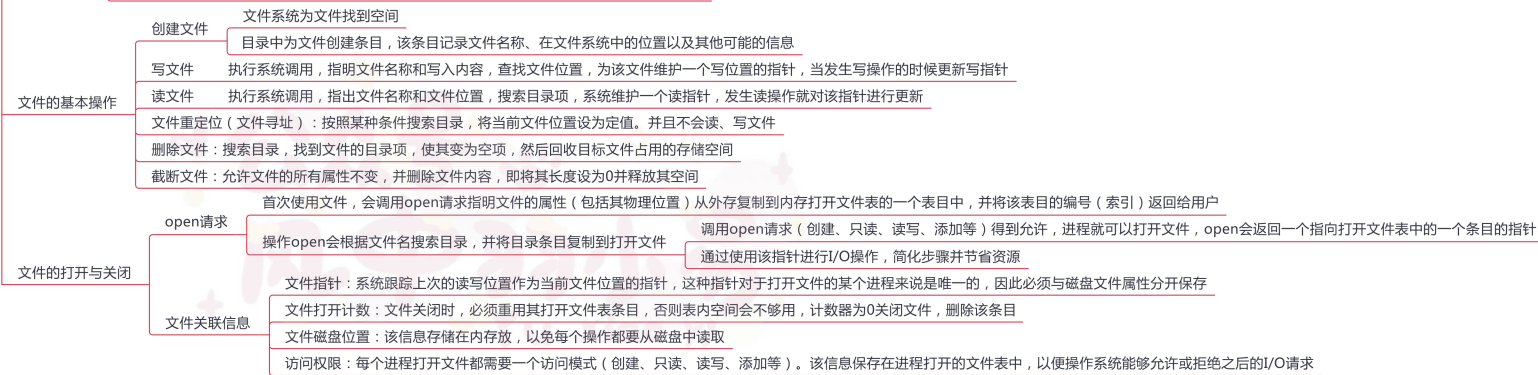
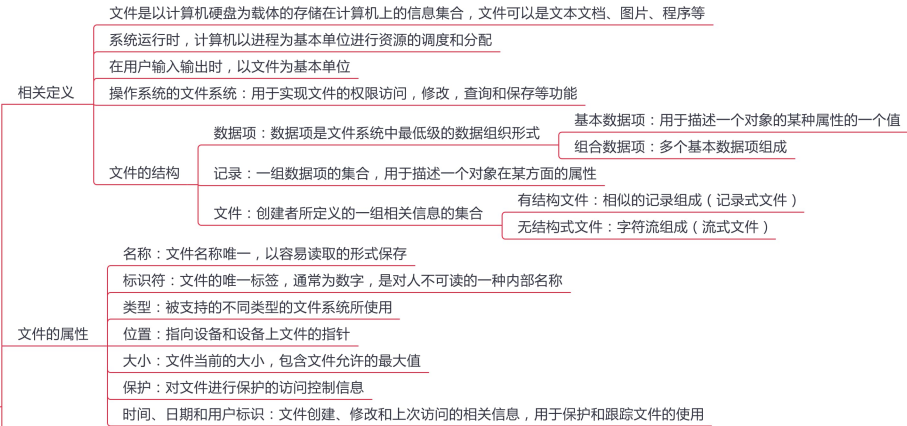


工作集

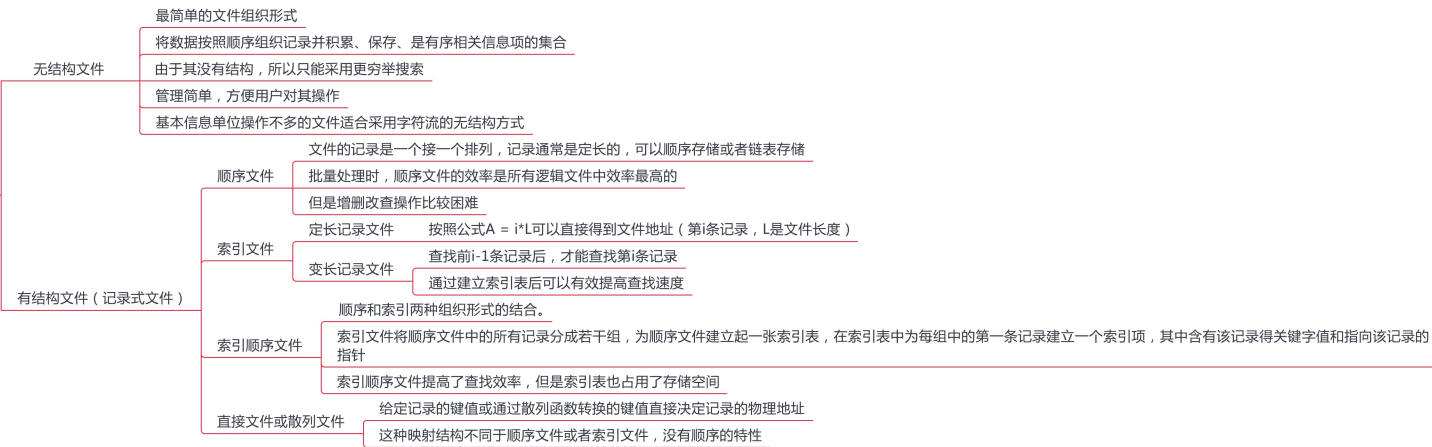


4.1文件系统基础（上）

文件的相关概念



文件的逻辑结构

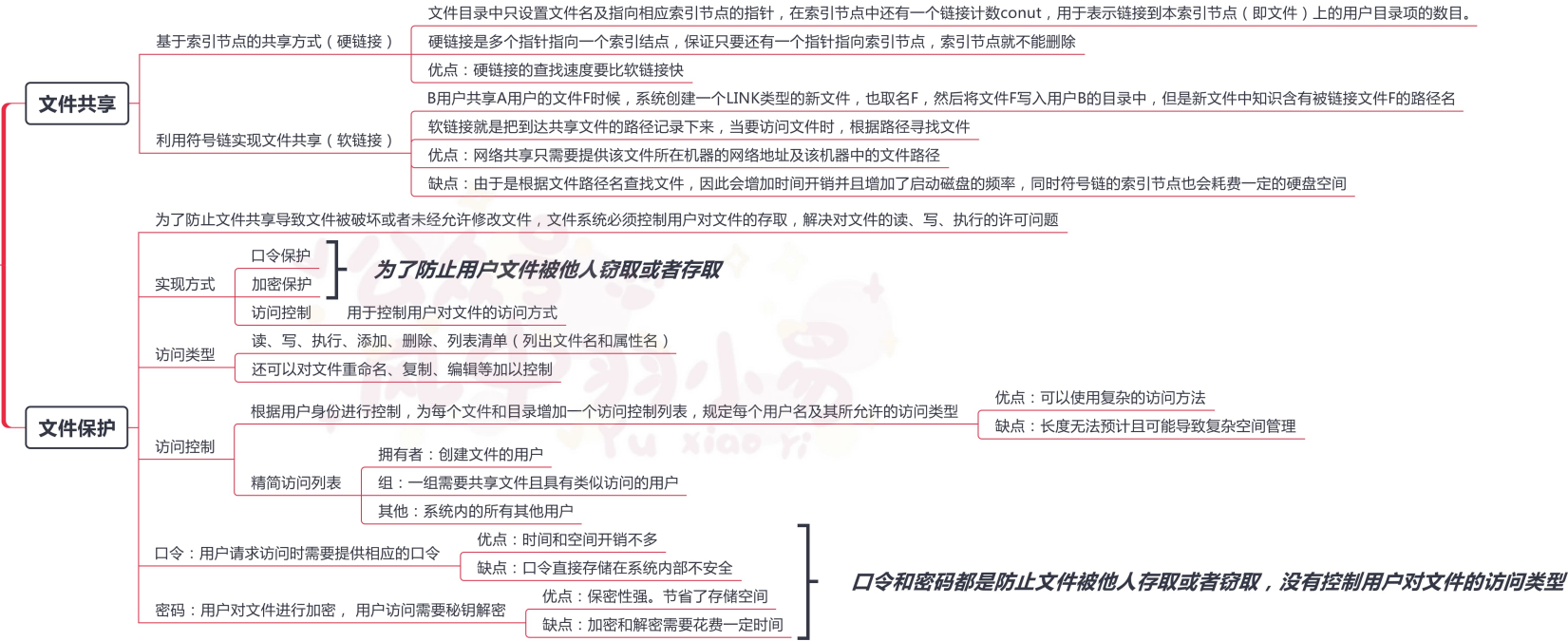


4.1文件系统基础（中）

目录结构



4.1文件系统基础（下）



4.2文件系统的实现



行列

4.3 磁盘组织与管理

磁盘结构

表面涂有磁性物质的金属或塑料构成的圆形盘片，通过一个称为磁头的导体线圈从磁盘读取数据。

磁盘盘面上的数据存储在在一组同心圆中，称为磁道

一个盘面上有上千个磁道，磁道又划分为几百个扇区，每个扇区固定存储大小，一份扇区称为一个盘块

磁盘地址用 柱面号-盘面号-扇区号(块号) 表示

磁盘分类

固定头磁盘：磁头相对于盘片的径向方向固定

活动头磁盘：每个磁道一个磁头，磁头可以移动

固定盘磁盘：磁头臂可以来回伸缩定位磁道，磁盘永久固定在磁盘驱动器内

可换盘磁盘：可以移动和替换

读写时间组成

寻找时间：活动头磁盘在读写信息前，将磁头移动到指定磁道所需要的时间（跨越n条磁道+启动磁臂）： $T_s = m \cdot n + s$

延迟时间：磁头定位到某一磁道扇区所需要的时间， $T_r = 1/2r$ （转速为r）

传输时间：从磁盘读出或向磁盘写入数据经过时间， $T_r = b / rN$

磁盘调度算法

先来先服务算法（FCFS）

按照进程请求访问磁盘的先后顺序进行调度

优点：公平 实现简单

缺点：适用于少量进程访问，如果进程过多算法更倾向于随机调度

最短寻找时间有限算法（SSTF）

选择调度处理的磁道是与当前磁头所在磁道距离最近的磁道

优点：性能强于先来先服务算法

缺点：容易产生饥饿现象

扫描(SCAN)算法

在磁头当前移动方向上选择与当前磁头所在的磁道距离最近的请求作为下一次服务对象

优点：寻道性能好，可以避免饥饿现象

缺点：对最近扫描过的区域不公平，访问局部性方面不如FCFS和SSTF好

循环扫描算法（C-SCAN）

磁头单向移动，回返时直接回到起始端，而不服任何请求

磁盘调度算法

LOOK与C-LOOK算法

在SCAN与C-SCAN算法的基础上规定了查看移动方向上是否有请求，如果没有就不会继续向前移动，而是直接改变方向（LOOK）或者直接回到第一个请求处（C-LOOK）

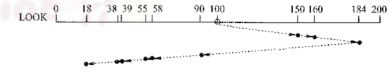


图 4.23-1 LOOK 磁盘调度算法

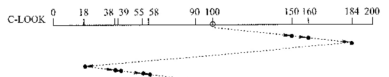


图 4.23-2 C-LOOK 磁盘调度算法

对盘面交替编号 原因：磁头在读/写一个物理块后，需要经过短暂的处理时间才能开始处理下一块

磁盘的管理

磁盘初始化

低级格式化：磁盘分扇区，为每个扇区采用特别的数据结构（头、数据区域、尾部组成），头部含有一些磁盘控制器所使用的信息

进一步格式化处理：磁盘分区，对物理分区进行逻辑格式化（创建文件管理系统），包括空闲和已分配的空间及一个初始为空的目录

引导块

计算机启动时运行自举程序，初始化CPU寄存器、设备控制器和内存等，然后启动操作系统

组局程序通常保存在ROM中，在ROM中保留很小的自举块，完整的自举程序保存在启动块上

拥有启动分区的磁盘称为启动磁盘或系统磁盘

坏块

无法使用的扇区

对于简单的磁盘，可以在逻辑格式化时（建立文件系统时）对整个磁盘进行坏块检查，标明哪些扇区是坏扇区，比如：在 FAT 表上标明

处理方式

简单磁盘：手动处理，对坏块进行标记，程序不会使用

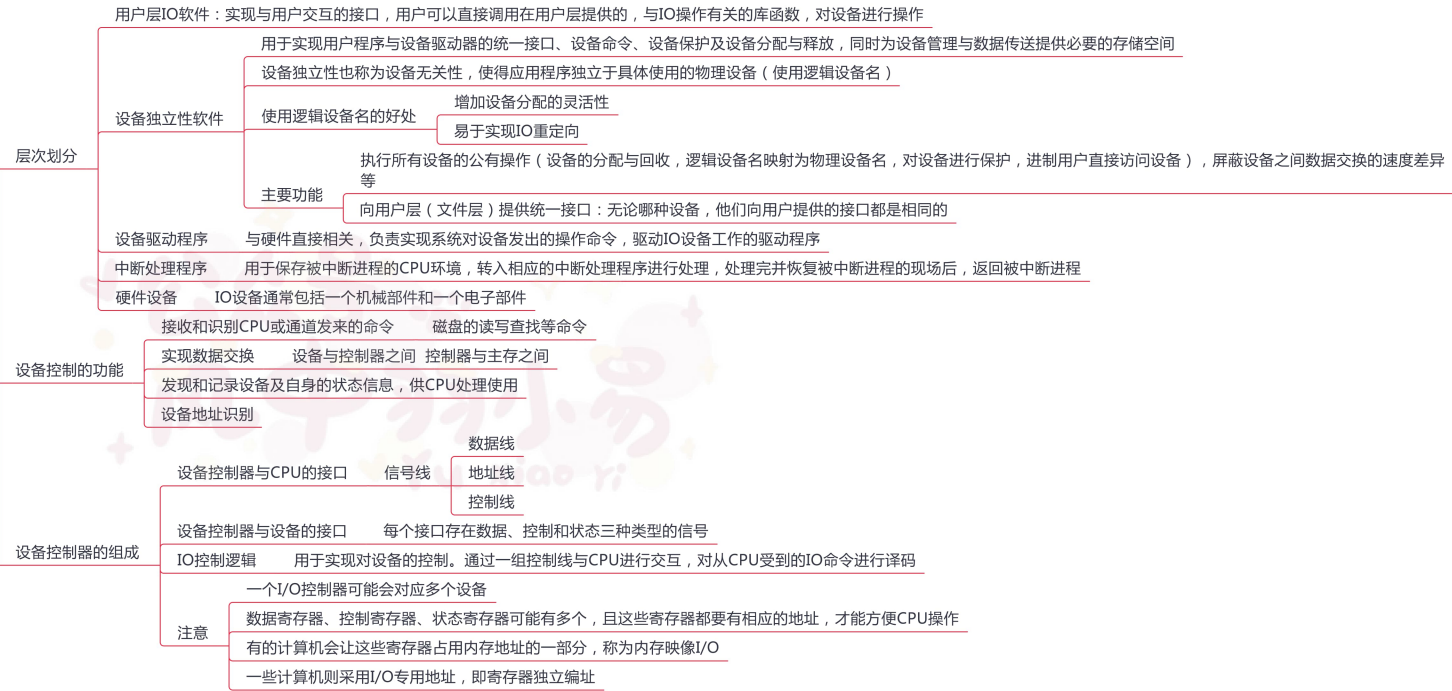
复杂磁盘：控制器维护一个磁盘坏块链表，同时将一些块作为备用，用于替代坏块（扇区备用）

5.1 IO管理概述（上）



5.1 IO管理概述（下）

I/O子系统的层次结构



5.2 IO核心子系统（上）



设备分配与回收

概述

根据用户IO请求分配设备，原则：充分发挥设备的使用效率，避免进程死锁

设备类型分类

独占式使用设备

设备只能互斥使用（打印机）

分时共享使用设备

通过分时共享来提高设备的利用率

SPOOLing方式使用设备

使用空间换时间，对IO设备进行批处理

设备分配的数据结构

设备控制表(DCT)

一个设备控制表表征一个设备，控制表中是设备的各项属性

控制器控制表(COCT)

COCT与DCT一一对应关系，DCT需要一个表项来表示控制器，即一个指向控制器控制表的指针

通道控制表(CHCT)

CHCT提供服务的那几个设备控制器

系统设备表(SDT)

记录已经连接到系统中的所有物理设备的情况

设备分配的策略

分配原则：充分发挥设备效率，避免进程死锁

分配方式

静态

系统一次性的把设备分配给相应作业，直到作业结束

优点：没有死锁问题

缺点：降低了设备使用率

动态

进程执行过程中根据执行需要进行分配

优点：提高了设备利用率

缺点：分配算法不当可能导致死锁

设备分配算法

先请求先分配

类似于先来先服务

优先级高者优先

独占设备一般使用静态分配，共享设备一般使用动态分配

5.2 IO核心子系统（下）

IO调度

- 设备分类
 - 独占设备——一个时段只能分配给一个进程（如打印机）
 - 共享设备——可同时分配给多个进程使用（如磁盘），各进程往往是宏观上同时共享使用设备，而微观上交替使用
 - 虚拟设备——采用 SPOOLing 技术将独占设备改造成虚拟的共享设备，可同时分配给多个进程使用（如采用 SPOOLing 技术实现的共享打印机）

设备分配的安全性

- 安全分配方式
 - 进程发出IO请求后便进入阻塞态，直到IO结束才被唤醒
 - 优点：设备分配安全
 - 缺点：CPU和IO设备串行工作
- 不安全分配方式
 - 进程发出IO请求后继续运行，需要时发出第二个，第三个IO请求
 - 优点：进程推进迅速
 - 缺点：可能产生死锁

逻辑设备名到物理设备名的映射

- 目的
 - 提高设备分配的灵活性和利用率
 - 实现IO重定向
 - 引入设备独立性
- 实现方法：引入逻辑设备表(LUT)，用来将逻辑设备名映射为物理设备名
- 建立方式
 - 整个系统设置一张LUT，所有设备分配情况都记录在这张表上（单用户设备）
 - 每个用户建立一张LUT，分别记录设备的分配情况

SPOOLING技术（假脱机技术）

- 目的
 - 缓解CPU 与IO 的速度差异矛盾
- 要实现SPOOLing 技术，必须要有多个道程序技术的支持
- 输入井和输出井
 - 输入井用来收容IO设备的数据
 - 输出井用来模拟输出时的磁盘
- 输入缓冲区和输出缓冲区
 - 输入缓冲区：暂存由输入设备送来的数据
 - 输出缓冲区：暂存从输出井送来的设备
- 输入进程和输出进程
 - 输入进程：模拟脱机输入时的外围控制机，将用户要求的数据从输入机通过输入缓冲区送到输入井中，当CPU需要数据，直接将输出井中的数据送入内存
 - 输出进程：模拟脱机输出时的外围控制机，把用户要求输出的数据先从内存送到输出井中，待输出设备空闲时，再将输出井中的数据经过输出缓冲区送到输出设备
- 特点
 - 提高了IO速度
 - 独占设备变成了共享设备
 - 实现了虚拟设备功能
- 通俗一点就是，如果设备被占用，我们就先把数据暂存一下，等到设备空闲了就把这些数据输送到设备中

高速缓存与缓冲区对比

相同点

都介于高速设备和低速设备之间

不同

存放数据

高速缓存：存放的是低速设备上的某些数据的复制数据

缓冲区：存放的是低速设备传递给高速设备的数据，这些数据在低速设备上不一定有备份，这些数据再从缓冲区传送到高速设备

目的

高速缓存：高速缓存存放的是高速设备经常要访问的数据，如高速缓存中数据不在，高速设备就要访问低速设备

高速设备和低速设备的通信都要经过缓冲区，高速设备永远不会去直接访问低速设备